



FPT UNIVERSITY

FPT UNIVERSITY – DA NANG CAMPUS SOFTWARE TESTING (SWT301)

LAB REPORT 1:

TESTING FUNDAMENTALS THROUGH EFFECTIVE PROMPTS

Full Name: Nguyễn Duy Phương

Student ID: DE190416

Class: SE20A11

Submission Date: 25/05/2026

AI Tool Used: ChatGPT

Personal GitHub Repository: https://github.com/duyphuong24/SU26_SWT301.git

Contents

1.	Task 1: Why is Testing Necessary?	4
1.1	Prompt	4
1.2	AI Response	4
1.2.1	Comparison Table of Basic Concepts	4
1.2.2	Cost of Late Testing (Multiplicative Increase in Cost)	5
1.3	Comments on the AI Response	6
2	Task 2: What is testing?	7
2.1	Prompt	7
2.2	AI Response	7
2.2.1	Definition of Software Testing	7
2.2.2	Comparison Between Testing and Debugging	8
2.2.3	Comparison Between Verification and Validation	8
2.3	Comments on the AI Response	9
3	Task 3: 7 Seven Principles of Software Testing	9
3.1	Prompt	9
3.2	AI Response	10
3.2.1	Principle 1: Exhaustive Testing is Impossible	10
3.2.2	Principle 2: Early Testing Saves Time and Money	10
3.2.3	Principle 3: Defects Cluster Together	11
3.3	Comments on the AI Response	11
4	Task 4: Test activities & roles	12
4.1	Prompt	12
4.2	AI Response	12
4.2.1	Test Planning	13
4.2.3	Test Analysis	13
4.2.4	Test Design	13

4.2.5	Test Implementation.....	14
4.2.6	Test Execution.....	14
4.2.7	Test Completion.....	14
4.3	Comments on the AI Response.....	14
5	Task 5: Essential Skills of a Software Tester (Custom Prompt Design for 2026)	15
5.1	Prompt Iteration Process (Initial and Improved Prompt Versions).....	15
5.2	AI Response After Prompt Optimization.....	16
5.2.1	Collaboration & Communication in Whole-Team Approach.....	16
5.2.2	Critical Thinking & Test Design Thinking.....	17
5.2.3	Independence of Testing.....	18
5.2.4	AI-Assisted Testing & Automation Skill.....	18
5.3	Comments on the Iterative Improvement Process	19
6	Conclusion.....	20
7	References.....	21

1. Task 1: Why is Testing Necessary?

1.1 Prompt

- **Role:** Bạn là một ISTQB-certified Test Manager với hơn 10 năm kinh nghiệm trong lĩnh vực kiểm thử phần mềm.
- **Context:** Tôi là sinh viên năm 2 đang học môn Software Testing và đang tìm hiểu chương Testing Fundamentals theo chuẩn ISTQB Foundation.
- **Task:** Hãy phân biệt 4 khái niệm sau:
 - Error
 - Defect
 - Failure
 - Root Cause

Với mỗi khái niệm:

- Đưa ra định nghĩa ngắn gọn
- Cho ví dụ thực tế trong hệ thống đặt vé xem phim CinePlex
- Giải thích mối liên hệ giữa các khái niệm
- **Format:** Trình bày dưới dạng bảng gồm 4 cột:
| Khái niệm | Định nghĩa | Ví dụ thực tế | Mối liên hệ |
- **Example:**
 - Developer viết sai điều kiện giảm giá -> tạo defect trong source code.
 - Người dùng thanh toán thất bại -> failure.

1.2 AI Response

1.2.1 Comparison Table of Basic Concepts

Concept	Definition	Real-world Example
Error (Mistake)	Hành động hoặc suy nghĩ	Developer hiểu sai yêu cầu

	sai của con người trong quá trình phát triển phần mềm.	“giảm 20% cho sinh viên” thành “giảm 20% cho mọi khách hàng”.
Defect (Bug / Fault)	Lỗi tồn tại trong mã nguồn, tài liệu hoặc thiết kế phần mềm.	Hệ thống tính sai giá vé do công thức giảm giá sai.
Failure	Hệ thống hoạt động không đúng với mong đợi khi thực thi.	Người dùng thanh toán vé nhưng hệ thống báo “Payment Failed” dù tài khoản đủ tiền.
Root Cause	Nguyên nhân gốc rễ dẫn đến error hoặc defect	Team không phân tích rõ yêu cầu nghiệp vụ trước khi coding.

1.2.2 Cost of Late Testing (Multiplicative Increase in Cost)

Nếu lỗi được phát hiện ở giai đoạn production, chi phí sửa lỗi sẽ cao hơn rất nhiều so với lúc phân tích hoặc thiết kế. Ví dụ:

- Nếu defect được phát hiện khi đang coding: chỉ cần sửa code.
- Nếu defect được phát hiện sau khi release:
 - Phải rollback hệ thống
 - Đền bù cho khách hàng
 - Mất uy tín doanh nghiệp
 - Tốn thời gian retest và redeploy

Theo ISTQB, kiểm thử sớm giúp giảm rủi ro và tiết kiệm chi phí đáng kể.

1.3 Comments on the AI Response

- Câu trả lời của AI khá rõ ràng và dễ hiểu đối với sinh viên mới học Testing Fundamentals. AI phân biệt đúng 4 khái niệm Error, Defect, Failure và Root Cause theo đúng flow thực tế trong phát triển phần mềm. Ví dụ về hệ thống CinePlex giúp em dễ liên hệ với dự án thực tế hơn.
- Tuy nhiên, phần giải thích mối quan hệ giữa các khái niệm còn hơi ngắn. Sau đó em đã tinh chỉnh prompt bằng cách yêu cầu “giải thích flow từ root cause đến failure” để AI trình bày logic rõ ràng hơn.
- Ngoài ra, em đã đối chiếu lại với tài liệu ISTQB Foundation để kiểm tra tính chính xác của định nghĩa.

2 Task 2: What is testing?

2.1 Prompt

- **Role:** Bạn là một ISTQB-certified Test Manager với hơn 10 năm kinh nghiệm trong lĩnh vực kiểm thử phần mềm.
- **Context:** Tôi đang ôn tập Chương 1 môn Software Testing và hay bị nhầm lẫn các thuật ngữ cơ bản.
- **Task:** Hãy đưa ra định nghĩa đầy đủ về “Kiểm thử phần mềm” dựa trên tiêu chuẩn ISTQB v4.0. Sau đó, tạo hai bảng so sánh phân biệt rõ ràng:
 - Testing vs Debugging
 - Verification vs Validation
- **Format:** Sắp xếp câu trả lời thành các phần riêng biệt:
 - Definition of Software Testing
 - Comparison Table 1
 - ComparisonTable2Để đảm bảo tính dễ đọc và dễ quét, hãy giữ tổng số từ trong câu trả lời dưới 350 từ.

2.2 AI Response

2.2.1 Definition of Software Testing

Theo ISTQB, Kiểm thử phần mềm là một quy trình bao gồm tất cả các hoạt động trong suốt vòng đời của phần mềm, bao gồm cả kiểm thử tĩnh (như xem xét tài liệu) và kiểm thử động (như thực thi mã).

Quy trình này bao gồm lập kế hoạch, chuẩn bị, đánh giá và xác minh các sản phẩm phần mềm và các sản phẩm công việc liên quan để xác định xem chúng có đáp ứng các yêu cầu đã chỉ định, chứng minh tính phù hợp với mục đích và xác định các lỗi hay không.

2.2.2 Comparison Between Testing and Debugging

Criteria	Testing	Debugging
Purpose	Nhằm xác định các khiếm khuyết và phát hiện lỗi trong hệ thống.	Xác định, phân tích và loại bỏ các lỗi trong mã nguồn.
Performed by	Người kiểm thử hoặc kỹ sư QA độc lập.	Các nhà phát triển hoặc lập trình viên đã viết mã.
Output	Báo cáo lỗi, nhật ký kiểm thử và kết quả kiểm thử.	Mã nguồn đã được sửa lỗi và cải tiến, hoạt động đúng như mong muốn.
Follow-up Action	Thực hiện lại các bài kiểm tra (kiểm tra xác nhận) để xác minh rằng lỗi đã được khắc phục thành công.	Hãy bàn giao phiên bản phần mềm đã được cập nhật và sửa lỗi cho nhóm kiểm thử.

2.2.3 Comparison Between Verification and Validation

Criteria	Verification	Validation
Core Question	“Liệu chúng ta đã xây dựng hệ thống một cách chính xác?”	“Liệu chúng ta đã xây dựng được hệ thống đúng đắn?”
Focus	Đánh giá các tài liệu, thông số kỹ thuật thiết kế	Đánh giá phần mềm đã hoàn thiện để xác định

	và mã nguồn để đảm bảo tuân thủ các yêu cầu và tiêu chuẩn kỹ thuật.	xem nó có đáp ứng nhu cầu người dùng và kỳ vọng kinh doanh trong môi trường thực tế hay không.
Example	Kiểm tra xem biểu mẫu thanh toán vé xem phim có chứa chính xác ba trường nhập liệu và giới hạn ký tự như đã quy định trong tài liệu SRS hay không.	Khách hàng đặt vé xem phim thành công và cảm thấy quy trình thanh toán thuận tiện, hiệu quả và thân thiện với người dùng.

2.3 Comments on the AI Response

- Trí tuệ nhân tạo đã phân biệt rõ ràng giữa hai cặp khái niệm thường bị nhầm lẫn: Kiểm thử so với Gỡ lỗi và Xác minh so với Thẩm định.
- Việc sử dụng các câu hỏi nổi tiếng của Barry Boehm về Kiểm chứng và Xác nhận đã giúp cho lời giải thích trở nên học thuật hơn và có cơ sở lý thuyết vững chắc hơn.
- Các bảng so sánh ngắn gọn, dễ xem lướt qua và phù hợp với thuật ngữ cũng như mục tiêu học tập của chứng chỉ ISTQB Foundation Level.

3 Task 3: 7 Seven Principles of Software Testing

3.1 Prompt

- **Role:** Bạn là chuyên gia kiểm thử phần mềm theo chuẩn ISTQB.
- **Context:** Nhóm phát triển dự án “BookMyMovie” (hệ thống đặt vé xem phim trực tuyến CinePlex) có một quan niệm sai lầm về kiểm thử phần mềm. Họ tin rằng việc kiểm thử phải loại bỏ 100% lỗi trước khi phát hành.
- **Task:** Hãy giải thích chi tiết ba nguyên tắc đầu tiên của kiểm thử phần mềm:
 - Việc kiểm tra toàn diện là không thể.
 - Kiểm tra sớm giúp tiết kiệm thời gian và tiền bạc.

- Các lỗi thường tập trung lại với nhau.
- **Format:** Đối với mỗi nguyên tắc, hãy cung cấp:
 - Một định nghĩa ngắn gọn (tối đa 30 từ)
 - Một ví dụ cụ thể hoặc hậu quả thực tiễn liên quan trực tiếp đến dự án BookMyMovie
 - Không sử dụng biểu tượng cảm xúc.

3.2 AI Response

3.2.1 Principle 1: Exhaustive Testing is Impossible

- **Definition:**
 - Việc kiểm tra mọi tổ hợp đầu vào và điều kiện tiên quyết có thể là bất khả thi, ngoại trừ đối với các hệ thống cực kỳ đơn giản.
- **Practical Consequence in the BookMyMovie Project:**
 - Tính năng đặt chỗ và phiếu giảm giá bao gồm hàng triệu kịch bản có thể xảy ra, bao gồm các tổ hợp chỗ ngồi VIP/thường, thời gian chiếu phim, chi nhánh rạp chiếu phim và phương thức thanh toán.
 - Thay vì thử nghiệm mọi trường hợp có thể xảy ra và làm chậm lịch phát hành, người kiểm thử nên áp dụng các kỹ thuật như Phân vùng tương đương và Phân tích giá trị biên để thiết kế một bộ trường hợp thử nghiệm đại diện được tối ưu hóa và dựa trên rủi ro.

3.2.2 Principle 2: Early Testing Saves Time and Money

- **Definition:**
 - Các hoạt động kiểm thử nên bắt đầu càng sớm càng tốt trong vòng đời phát triển phần mềm để giảm thiểu rủi ro và hỗ trợ hiệu quả các mục tiêu dự án.
- **Practical Consequence in the BookMyMovie Project:**

- Khi các chuyên viên phân tích nghiệp vụ soạn thảo các yêu cầu hoàn tiền vé trực tuyến, người kiểm thử nên tham gia ngay vào việc xem xét yêu cầu thông qua các hoạt động kiểm thử tĩnh.
- Việc phát hiện logic hoàn tiền không nhất quán ở giai đoạn lập tài liệu có thể chỉ cần chỉnh sửa nhỏ, trong khi việc phát hiện cùng một vấn đề sau khi triển khai có thể đòi hỏi việc thiết kế lại và phát triển lại hệ thống thanh toán phía sau tốn kém.

3.2.3 Principle 3: Defects Cluster Together

- **Definition:**
 - Một số lượng nhỏ các mô-đun thường chứa phần lớn các lỗi được phát hiện trong quá trình thử nghiệm hoặc các sự cố xảy ra trong môi trường sản xuất.
- **Practical Consequence in the BookMyMovie Project:**
 - Trong nhiều chu kỳ thử nghiệm, “Công thanh toán tích hợp và mô-đun giảm giá kết hợp” luôn tạo ra hơn 80% lỗi của hệ thống, phản ánh nguyên tắc Pareto (quy tắc 80/20).
 - Hiểu được nguyên tắc này giúp nhóm thử nghiệm tập trung nỗ lực và nguồn lực kiểm thử hồi quy vào các mô-đun có rủi ro cao và dễ phát sinh lỗi thay vì phân bổ nỗ lực đồng đều cho tất cả các tính năng, bao gồm cả các mô-đun đơn giản hơn như trang tin tức phim.

3.3 Comments on the AI Response

- Việc cung cấp bối cảnh dự án cụ thể như “BookMyMovie” đã làm cho câu trả lời trở nên thực tế và thiết thực hơn so với các ví dụ chung chung trong sách giáo khoa.
- Các ví dụ phản ánh sát với các hoạt động kiểm thử thực tế được thực hiện bởi các kỹ sư đảm bảo chất lượng phần mềm trong môi trường công nghiệp.

- Trí tuệ nhân tạo cũng kết nối các nguyên tắc kiểm thử với các kỹ thuật kiểm thử thực tiễn và các chiến lược kiểm thử dựa trên rủi ro, điều này đã cải thiện cả chiều sâu kỹ thuật và tính ứng dụng của lời giải thích.

4 Task 4: Test activities & roles

4.1 Prompt

- **Role:** Đóng vai một Test Manager có hơn 12 năm kinh nghiệm trong các dự án enterprise.
- **Context:** Tôi cần giải thích quy trình kiểm thử tiêu chuẩn ISTQB cho các thành viên mới trong nhóm để họ hiểu cách các hoạt động kiểm thử được tổ chức và phối hợp trong các dự án thực tế.
- **Task:** Hãy mô tả rõ bảy hoạt động chính trong Quy trình Kiểm thử ISTQB, bao gồm:
 - Test Planning
 - Test Monitoring & Control
 - Test Analysis
 - Test Design
 - Test Implementation
 - Test Execution
 - Test Completion
- **Format:** Trình bày câu trả lời dưới dạng danh sách được đánh số từ 1 đến 7. Mỗi hoạt động cần được giải thích không quá hai câu và phải đề cập rõ ràng đến sản phẩm đầu ra hoặc phần mềm kiểm thử liên quan được tạo ra trong hoạt động đó.

4.2 AI Response

- Chào mừng đến với nhóm QA. Trước khi đi sâu vào các nhiệm vụ kiểm thử kỹ thuật, tôi luôn nhắc nhở các thành viên mới rằng không có quy trình kiểm thử chung nào phù hợp với mọi dự án phần mềm. Quy trình kiểm thử phải thích ứng dựa trên bối

cảnh dự án như mô hình vòng đời phát triển phần mềm, rủi ro kinh doanh, cấp độ kiểm thử, thời hạn, quy mô nhóm và độ phức tạp kỹ thuật.

- Dưới đây là tổng quan về bảy hoạt động kiểm thử chính và các sản phẩm kiểm thử chính mà nhóm của tôi thường tạo ra trong suốt vòng đời dự án. **Test Planning**

- Tôi bắt đầu bằng việc xác định phạm vi thử nghiệm, mục tiêu, chiến lược, phân bổ nguồn lực và các rủi ro tiềm ẩn có thể ảnh hưởng đến tiến độ dự án.
- Kết quả chính của giai đoạn này là Kế hoạch Thử nghiệm, bao gồm lịch trình, trách nhiệm, tiêu chí đầu vào và đầu ra, và các phương pháp quản lý rủi ro. **Test Monitoring and Control**

- During project execution, I continuously compare actual testing progress against the original plan using testing metrics and status indicators.
- This activity produces **Test Progress Reports** and corrective control actions used to adjust priorities, schedules, or team assignments when necessary.

4.2.1 Test Analysis

- Ở giai đoạn này, người kiểm thử sẽ xem xét kỹ lưỡng cơ sở kiểm thử như đặc tả yêu cầu, câu chuyện người dùng, tài liệu kiến trúc và báo cáo phân tích rủi ro để xác định những gì cần được kiểm thử.
- Kết quả chính là các Điều kiện Kiểm thử được ghi lại và mối quan hệ truy vết giữa các yêu cầu và mục tiêu kiểm thử.

4.2.2 Test Design

- Sau khi xác định được các mục tiêu kiểm thử, nhóm sẽ thiết kế các kịch bản kiểm thử chi tiết, chuẩn bị dữ liệu kiểm thử và xác định môi trường kiểm thử cần thiết cùng các công cụ hỗ trợ.
- Các sản phẩm chính bao gồm: Trường hợp kiểm thử, Đặc tả dữ liệu kiểm thử và các kịch bản kiểm thử được ưu tiên. **Test Implementation**

- In this phase, I ensure that all required testing resources, environments, and execution procedures are fully prepared before actual testing begins.
- The outputs include **Test Procedures**, organized **Test Suites**, and the **Test Execution Schedule** used for coordinated execution activities.

4.2.3 Test Execution

- Nhóm của tôi thực hiện các trường hợp kiểm thử thủ công hoặc tự động theo lịch trình đã được phê duyệt và so sánh kết quả thực tế với kết quả mong đợi bằng cách sử dụng các tiêu chí kiểm thử đã được định nghĩa.
- Hoạt động này liên tục tạo ra Nhật ký kiểm thử, bản ghi thực thi và Báo cáo lỗi chi tiết bất cứ khi nào phát hiện ra sự bất thường.

4.2.4 Test Completion

- Vào cuối dự án hoặc chu kỳ thử nghiệm, tôi xem xét kết quả thử nghiệm, tóm tắt những bài học kinh nghiệm và xác nhận việc khắc phục các lỗi đã được xử lý và các vấn đề đang chờ giải quyết.
- Sản phẩm cuối cùng là Báo cáo Tóm tắt Thử nghiệm, cung cấp cho các bên liên quan đánh giá tổng thể về các hoạt động thử nghiệm và chất lượng sản phẩm.

4.3 Comments on the AI Response

- Cách tiếp cận kể chuyện theo ngôi thứ nhất đã làm cho quy trình kiểm thử ISTQB trở nên hấp dẫn và dễ hiểu hơn so với lời giải thích thuần túy lý thuyết.
- Trí tuệ nhân tạo (AI) đã kết nối rõ ràng từng hoạt động kiểm thử với sản phẩm kiểm thử tương ứng, đây là một khái niệm quan trọng thường được nhấn mạnh trong các kỳ thi ISTQB Foundation Level.
- Phản hồi cũng phản ánh các trách nhiệm và hoạt động phối hợp thực tế thường được thực hiện bởi các Quản lý Kiểm thử giàu kinh nghiệm trong các dự án phần mềm chuyên nghiệp.

5 Task 5: Essential Skills of a Software Tester (Custom Prompt Design for 2026)

5.1 Prompt Iteration Process (Initial and Improved Prompt Versions)

- Lần lặp 1 – Lời nhắc ban đầu không có dữ liệu:

“What are the most important skills for a professional Software Tester?”
- Đánh giá lần lặp 1:
 - Trí tuệ nhân tạo (AI) đã đưa ra một câu trả lời rất chung chung và thiếu chiều sâu cũng như tính ứng dụng thực tiễn trong ngành kiểm thử phần mềm hiện đại.
 - Hầu hết các ý tưởng được trả về đều là những khái niệm lặp đi lặp lại thường thấy trong các cuộc thảo luận trực tuyến lỗi thời từ đầu những năm 2020, chẳng hạn như “tìm lỗi”, “học Selenium” hoặc “có kỹ năng giao tiếp”.
 - Câu trả lời không đề cập đến các xu hướng mới nổi như kiểm thử có sự hỗ trợ của AI, hợp tác nhóm toàn diện theo phương pháp Agile, tích hợp CI/CD, hoặc tầm quan trọng ngày càng tăng của tư duy phản biện và kiểm thử dựa trên rủi ro trong môi trường QA hiện đại.
 - Ngoài ra, câu trả lời được cấu trúc kém và không kết nối rõ ràng các kỹ năng cần thiết với chương trình giảng dạy của chứng chỉ ISTQB Foundation, khiến việc sử dụng trực tiếp trong báo cáo học thuật trở nên khó khăn.

Lần lặp 2 – Tối ưu hóa lời nhắc bằng cách sử dụng cấu trúc R-C-T-F-E

- **Role:**

Bạn là Giám đốc Công nghệ (CTO) kiêm Chuyên gia Đào tạo Cao cấp được chứng nhận ISTQB, chuyên về đảm bảo chất lượng phần mềm và các phương pháp kiểm thử dựa trên trí tuệ nhân tạo.
- **Context:**
 - Năm nay là năm 2026, ngành công nghiệp kiểm thử phần mềm đang nhanh chóng phát triển theo hướng kiểm thử hỗ trợ bởi trí tuệ nhân tạo

(AI), tự động hóa thông minh, tích hợp DevOps và phương pháp Agile toàn nhóm.

- Các chuyên viên kiểm thử phần mềm hiện đại không chỉ được kỳ vọng thực hiện các trường hợp kiểm thử mà còn phải cộng tác với các nhà phát triển, phân tích rủi ro, hiểu logic nghiệp vụ và sử dụng hiệu quả các công cụ AI để nâng cao chất lượng phần mềm.

- **Task:**

- Phân tích và giải thích các nhóm kỹ năng thiết yếu mà một chuyên gia kiểm thử phần mềm chuyên nghiệp cần phát triển để duy trì khả năng cạnh tranh và thích ứng trong thị trường công nghệ năm 2026.
- Dựa trên Mục 1.5 của giáo trình ISTQB Foundation, bao gồm các kỹ năng hợp tác, tư duy phản biện, tính độc lập trong kiểm thử và khả năng kiểm thử có sự hỗ trợ của trí tuệ nhân tạo (AI).

- **Format:**

- Sắp xếp câu trả lời thành các phần riêng biệt rõ ràng (5.2.1 → 5.2.4).
- Đối với mỗi loại kỹ năng, hãy cung cấp:
 1. Giải thích ngắn gọn về kỹ năng và tầm quan trọng của nó theo các khái niệm của ISTQB.
 2. Những hành động thực tiễn mà sinh viên Đại học FPT có thể áp dụng ngay trong quá trình học tập để nâng cao kỹ năng đó.
- Sử dụng văn phong học thuật trang trọng phù hợp với báo cáo kiểm thử phần mềm của trường đại học.

5.2 AI Response After Prompt Optimization

5.2.1 Collaboration & Communication in Whole-Team Approach

- **Core Explanation:**

- Theo Mục 1.5 của giáo trình ISTQB Foundation, chất lượng phần mềm không nên được xem là trách nhiệm riêng của nhóm kiểm thử. Thay vào đó, Phương pháp làm việc nhóm toàn diện nhấn mạnh rằng các nhà phát

triển, người kiểm thử, nhà phân tích kinh doanh và chủ sở hữu sản phẩm phải cộng tác liên tục trong suốt vòng đời phát triển phần mềm.

- Do đó, một người kiểm thử chuyên nghiệp cần có kỹ năng giao tiếp và kỹ năng mềm tốt để báo cáo lỗi một cách khách quan, thảo luận về rủi ro một cách mang tính xây dựng và tránh những xung đột không cần thiết giữa các thành viên trong nhóm. Giao tiếp hiệu quả cũng giúp đảm bảo rằng những hiểu lầm về yêu cầu và logic kinh doanh được xác định sớm trước khi bắt đầu triển khai.

- **Practical Actions for Students:**

- Trong các dự án làm việc nhóm, sinh viên nên thực hành các hoạt động đánh giá đồng nghiệp như cùng nhau xem xét mã nguồn, tài liệu yêu cầu hoặc thiết kế giao diện người dùng thay vì làm việc riêng lẻ.
- Khi phát hiện lỗi trong công việc của đồng đội, sinh viên nên giải thích rõ vấn đề, cung cấp các bước kiểm thử có thể tái hiện và giao tiếp một cách tôn trọng và mang tính xây dựng thay vì tập trung vào chỉ trích hoặc đổ lỗi.

5.2.2 Critical Thinking & Test Design Thinking

- **Core Explanation:**

- Chương trình giảng dạy ISTQB nhấn mạnh tư duy phản biện là một trong những khả năng thiết yếu nhất đối với người kiểm thử phần mềm. Người kiểm thử không nên chỉ tin tưởng vào các thông số kỹ thuật hoặc cho rằng hệ thống luôn hoạt động chính xác trong điều kiện bình thường.
- Thay vào đó, họ phải phân tích sản phẩm từ nhiều góc độ, đặt câu hỏi về các giả định, xác định các điểm yếu logic tiềm ẩn và thiết kế các kịch bản kiểm thử nhắm vào các trường hợp ngoại lệ, đầu vào bất thường và các tình huống rủi ro cao mà nhà phát triển có thể bỏ qua.

- **Practical Actions for Students:**

- Trong các dự án đại học như ứng dụng web Java hoặc đồ án tốt nghiệp, sinh viên nên tránh chỉ tập trung vào các luồng thực thi thành công (“Đường dẫn hạnh phúc”).
- Họ nên chủ động kiểm tra dữ liệu không hợp lệ, hành vi người dùng không mong đợi và các điều kiện biên khắc nghiệt trong khi phân tích logic nghiệp vụ một cách cẩn thận thông qua sơ đồ, quy trình làm việc và các kỹ thuật tư duy dựa trên rủi ro.

5.2.3 Independence of Testing

- **Core Explanation:**

- Giáo trình ISTQB Foundation giải thích rằng một mức độ độc lập nhất định trong kiểm thử sẽ cải thiện đáng kể hiệu quả phát hiện lỗi vì người đánh giá độc lập ít bị ảnh hưởng bởi các giả định và thành kiến nhận thức của nhà phát triển
- Tính độc lập trong kiểm thử có thể tồn tại ở nhiều cấp độ, từ việc các nhà phát triển tự kiểm thử mã của mình đến các nhóm QA nội bộ độc lập và thậm chí cả các tổ chức kiểm thử bên ngoài. Mức độ độc lập càng cao, khả năng phát hiện ra các lỗi chưa được chú ý một cách khách quan càng lớn.

- **Practical Actions for Students:**

- Sinh viên nên hình thành thói quen chuẩn bị các trường hợp kiểm thử và kết quả mong đợi trước khi bắt đầu triển khai để tách biệt tư duy kiểm thử khỏi tư duy lập trình.
- Sau khi hoàn thành bài tập hoặc dự án phần mềm, họ nên trao đổi dự án với bạn cùng lớp để thực hiện các hoạt động kiểm thử chéo nhằm nâng cao tính khách quan và phát hiện ra các lỗi mà các nhà phát triển ban đầu có thể bỏ sót.

5.2.4 AI-Assisted Testing & Automation Skill

- **Core Explanation:**

- Trong bối cảnh công nghệ năm 2026, Trí tuệ Nhân tạo (AI) ngày càng được tích hợp vào các hoạt động kiểm thử phần mềm như tạo kiểm thử tự động, tạo dữ liệu kiểm thử thông minh, dự đoán lỗi và kiểm thử hồi quy dựa trên AI.
- Do đó, các chuyên gia kiểm thử phần mềm hiện đại cần nhiều hơn kiến thức kiểm thử thủ công truyền thống. Họ phải hiểu cách sử dụng các nền tảng kiểm thử hỗ trợ AI, áp dụng hiệu quả các kỹ thuật kỹ thuật nhanh chóng và tự đánh giá độ tin cậy, tính công bằng và độ chính xác của các hệ thống dựa trên AI.
- **Practical Actions for Students:**
 - Sinh viên nên chủ động học các kỹ thuật kỹ thuật tạo kịch bản kiểm thử nâng cao để tạo ra các kịch bản kiểm thử và dữ liệu kiểm thử tổng hợp bằng các công cụ AI.
 -
 - Ngoài ra, họ nên nghiên cứu các khái niệm cơ bản về Học máy, thử nghiệm với các quy trình tự động hóa CI/CD và thực hành sử dụng các công cụ như GitHub Actions và Jenkins trong các dự án phát triển phần mềm của trường đại học để chuẩn bị cho các yêu cầu của ngành công nghiệp trong tương lai.

5.3 Comments on the Iterative Improvement Process

- Việc áp dụng cấu trúc gợi ý R-C-T-F-E đã cải thiện đáng kể chất lượng, chiều sâu và tính tổ chức của phản hồi do AI tạo ra so với phiên bản ban đầu không có dữ liệu huấn luyện.
- Gợi ý được tối ưu hóa đã hướng dẫn AI đưa ra cách hiểu chính xác hơn nhiều về Mục 1.5 trong chương trình giảng dạy ISTQB Foundation, đặc biệt là về tư duy phản biện, kỹ năng hợp tác, tính độc lập trong kiểm thử và phương pháp tiếp cận Toàn nhóm trong môi trường Agile.

- Ngoài ra, gợi ý thứ hai đã chuyển hướng thành công cuộc thảo luận từ các quan điểm kiểm thử lỗi thời sang các xu hướng hiện đại của ngành công nghiệp năm 2026, bao gồm kiểm thử có sự hỗ trợ của AI, tự động hóa và các thực tiễn CI/CD.
- Việc tổ chức phản hồi thành các tiểu mục được đánh số rõ ràng (5.2.1 → 5.2.4) cũng đã cải thiện cấu trúc học thuật của báo cáo, làm cho nội dung chuyên nghiệp hơn, được kết nối logic hơn và dễ dàng hơn cho giảng viên xem xét và đánh giá.

6 Conclusion

Tổng kết lại, bài thực hành Lab 1 đã cung cấp một phương pháp tiếp cận toàn diện trong việc lĩnh hội các nền tảng kiểm thử phần mềm (Testing Fundamentals) thông qua kỹ thuật Kỹ sư Câu lệnh (Prompt Engineering). Quá trình hoàn thiện 5 nhiệm vụ không chỉ giúp củng cố lý thuyết chuyên ngành mà còn rèn luyện tư duy khai thác công cụ Trí tuệ Nhân tạo một cách có hệ thống

- **Regarding Testing Fundamentals:**

Việc phân tích sâu chuỗi khái niệm Error – Defect – Failure cùng nguyên nhân gốc rễ (Root Cause) đã định hình một tư duy đảm bảo chất lượng (QA) bài bản. Việc phân định rạch ròi giữa Testing và Debugging, hay Verification và Validation, kết hợp với việc nghiên cứu 7 nguyên tắc kiểm thử cốt lõi của ISTQB, đã làm nổi bật nguyên lý rằng kiểm thử không chỉ là tìm lỗi ở giai đoạn cuối mà là một quá trình liên tục, phụ thuộc vào bối cảnh (context-dependent). Việc hệ thống hóa 7 giai đoạn trong quy trình kiểm thử (Test Process) cũng giúp thiết lập cái nhìn tổng quan về cách thức quản lý, thiết kế và thực thi kịch bản kiểm thử trong các dự án thực tế.

- **Regarding Prompt Engineering and AI Usage:**

Bài thực hành đã chứng minh tính hiệu quả của công thức R-C-T-F-E (Role – Context – Task – Format – Example) trong việc định hướng và giới hạn không gian phản hồi của mô hình ngôn ngữ lớn. Thông qua việc so sánh các phương pháp zero-shot và few-shot, cũng như quá trình liên tục tinh chỉnh (iterate) câu lệnh, em nhận thấy rằng chất lượng đầu ra của AI tỷ lệ thuận với tính cụ thể của bối cảnh và cấu trúc định dạng được yêu cầu. Hơn nữa, việc bắt buộc đối chiếu kết quả của AI với giáo trình chính thống đã mài giũa tư duy phản biện, giúp nhận diện và kiểm soát hiện tượng "ảo giác" (hallucination) thường gặp ở các công cụ AI.

7 References

1. Black, R., van Veenendaal, E., & Graham, D. *Foundations of Software Testing: ISTQB Certification* (4th ed.). Cengage Learning, 2019.
2. International Software Testing Qualifications Board. *Certified Tester Foundation Level (CTFL) Syllabus Version 4.0*. ISTQB Official Publication, 2023.
3. [Prompt Engineering Guide](#). Accessed May 2026. This resource was used to study structured prompting techniques such as role-based prompting, context specification, and iterative prompt refinement for Large Language Models (LLMs).
4. Myers, G. J., Sandler, C., & Badgett, T. *The Art of Software Testing* (3rd ed.). John Wiley & Sons, 2011.
5. [Atlassian Software Testing Overview](#). Referenced for Agile testing concepts, QA collaboration workflows, and modern software testing practices.